Lumina
Deep Search Across All Extensions

Project Team

Sahil Anwar	22P-9141
Shayan Rasool	22P-9157
Azeem Ahmad	22P-9336

Session 2022–2026

Supervised by

Dr. Ali Sayyed



Department of Computer Science

National University of Computer and Emerging Sciences
Peshawar, Pakistan

September, 2025

Student's Declaration

We hereby declare that the thesis submitted is our own, original work, completed independently without any unauthorized assistance. Only the sources and resources explicitly cited have been utilized. All excerpts, whether quoted directly or paraphrased, have been properly acknowledged. If generative AI tools were used, we have clearly indicated the product name, manufacturer, software version, and the specific purpose (e.g., language enhancement, systematic research). We take full responsibility for the selection, application, and interpretation of any AI-generated content incorporated into this work. A list of prompts along with corresponding page numbers where they are used is included in the appendix.

Shayan Rasool Signature: _____

Sahil Anwar Signature: _____

Azeem Khan Signature: _____

_____

Verified by Plagiarism Cell Officer
Dated: _____

Certificate of Approval



The Department of Computer Science, National University of Computer and Emerging Sciences accepts this thesis titled **"Lumina: Deep Search Across All Extensions"**, submitted by **Shayan Rasool (22P-9157)**, **Sahil Anwar (22P-9141)**, and **Azeem Khan (22P-9336)**, in its current form, and it is satisfying the dissertation requirements for the award of Bachelors Degree in Computer Science.

Supervisor
Dr. Ali Sayyed Signature: _____

Mr.Raiz Nawab
FYP Coordinator
National University of Computer and Emerging Sciences, Peshawar

---

Dr.Qasim Jaan
HoD of Department of Computer Science
National University of Computer and Emerging Sciences

Acknowledgements

This project, **Lumina**, would not have been possible without the guidance and support of many individuals. We sincerely thank our supervisor, **Dr.Ali Sayyed**, for his continuous mentorship, valuable feedback, and careful review throughout the project.

We are also grateful to **Dr.Nouman Azam** for his encouragement and insightful suggestions.

Special thanks to our FYP Coordinator, **Mr.Riaz Nawab**, and **Mr.Abdul Rehman** for their administrative support and coordination. We also appreciate all faculty members of the Department of Computer Science for their guidance and time.

We are deeply grateful to our **parents** and **siblings** for their unconditional love, prayers, and constant support, which made this achievement possible.

Finally, we extend our gratitude to **the National University of Computer and Emerging Sciences** for providing us the opportunity and platform to bring **Lumina** to life.

Shayan Rasool
Sahil Anwar
Azeem Khan

Abstract

The rapid growth of digital content across multiple file formats has made traditional keyword-based search systems insufficient for efficient information retrieval. These systems often struggle to understand context, handle heterogeneous data types, and maintain user privacy.

This project presents **Lumina: Deep Search Across All Extensions**, an AI-powered offline multimodal search system designed to perform semantic retrieval across diverse file types including documents, images, and audio files. Instead of relying on a single model, Lumina uses an **orchestration-based architecture** that integrates multiple specialized AI models to extract, process, and understand content from different modalities. The extracted information is converted into embeddings and indexed using FAISS for fast and accurate similarity-based search.

The system enhances retrieval quality through a modular pipeline that includes OCR for image-based text extraction, text embedding models for semantic understanding, and a unified orchestration layer that coordinates all components. This design ensures flexibility, scalability, and improved accuracy across heterogeneous data sources while remaining fully offline for privacy protection.

Lumina provides an intuitive desktop interface that supports semantic search, contextual result ranking, and efficient browsing of locally stored data. The project demonstrates how orchestrated AI models can be combined into a single unified system to enable intelligent, fast, and privacy-preserving deep search across all file extensions.

Contents

1 Introduction **1**
- 1.1 Overview . . . 1
- 1.2 Purpose of the Investigation . . . 2
- 1.3 Problem Being Investigated . . . 2
- 1.4 Background and Importance . . . 2
- 1.5 Thesis and General Approach . . . 2
- 1.6 Criteria for Success . . . 2
  - 1.6.1 Project Scope . . . 2

2 Review of Literature **4**
- 2.1 Introduction . . . 4
- 2.2 Existing Work . . . 4
- 2.3 Tabular Analysis of Literature . . . 4
- 2.4 Conclusion . . . 4

3 Project Vision **6**
- 3.1 Problem Statement . . . 6
- 3.2 Business Opportunity . . . 6
- 3.3 Objectives . . . 6
- 3.4 Project Scope . . . 6
- 3.5 Constraints . . . 7
- 3.6 Stakeholders Description . . . 7
  - 3.6.1 Stakeholders Summary . . . 7
  - 3.6.2 Key High-Level Goals and Problems of Stakeholders . . . 7

4 Software Requirements Specification **8**
- 4.1 Introduction . . . 8
  - 4.1.1 Purpose . . . 8
  - 4.1.2 Document Conventions . . . 8
  - 4.1.3 Intended Audience and Reading Suggestions . . . 8
  - 4.1.4 Product Scope . . . 9
  - 4.1.5 References . . . 9
- 4.2 Overall Description . . . 9
  - 4.2.1 Product Perspective . . . 9
  - 4.2.2 Product Functions . . . 9
  - 4.2.3 User Classes and Characteristics . . . 10
  - 4.2.4 Operating Environment . . . 10
  - 4.2.5 Design and Implementation Constraints . . . 10
  - 4.2.6 User Documentation . . . 10

4.2.7 Assumptions and Dependencies . . . 10
4.3 External Interface Requirements . . . 11
4.3.1 User Interfaces . . . 11
4.3.2 Hardware Interfaces . . . 11
4.3.3 Software Interfaces . . . 11
4.3.4 Communication Interfaces . . . 11
4.4 System Features . . . 11
4.4.1 Feature 1: Multimodal Search Engine . . . 11
4.4.2 Feature 2: Lumina-OCR Enhancer . . . 12
4.4.3 Feature 3: Fusion Embedding Model . . . 12
4.4.4 Feature 4: Keyword Heatmap . . . 12
4.4.5 Feature 5: Summarization . . . 12
4.5 Other Nonfunctional Requirements . . . 12
4.5.1 Performance . . . 12
4.5.2 Safety . . . 13
4.5.3 Security . . . 13
4.5.4 Software Quality Attributes . . . 13
4.5.5 Business Rules . . . 13
4.6 Other Requirements . . . 13
4.7 Appendices . . . 13
4.7.1 Appendix A: Glossary . . . 13
4.7.2 Appendix B: Analysis Models . . . 14
4.7.3 Appendix C: To Be Determined . . . 14

5 Iteration Plan **15**
5.1 Midterm FYP-1 (August 2025 – October 2025) . . . 15
5.2 Final FYP-1 (November 2025 – January 2026) . . . 16
5.3 Midterm FYP-2 (February 2026 – April 2026) . . . 16
5.4 Final FYP-2 (May 2026 – July 2026) . . . 17

6 Iteration 1 **19**
6.1 Structural Design . . . 19
6.2 Activity Diagram . . . 20
6.3 Context Diagram . . . 21
6.4 ER Diagram . . . 22
6.5 Flow Diagram . . . 23
6.6 Sequence Diagram . . . 24
6.7 Use Case Diagram . . . 25
6.8 Data Structure Design . . . 26
6.8.1 Storage of Embeddings (Vectors) . . . 26
6.8.2 Representation of Chunks (List of Dictionaries) . . . 27
6.8.3 Ranking of Search Results (Priority Queue / Heap) . . . 27
6.9 Algorithm Design . . . 28
6.9.1 Lumina-OCR Enhancer . . . 28
6.9.2 Fusion Embedding Model . . . 29

7 Iteration 2 **32**
7.1 Structural Design . . . 32
7.2 Activity Diagram . . . 33
7.3 Call Graph . . . 34
7.4 ER Diagram . . . 35
7.5 Flow Diagram . . . 36
7.6 Sequence Diagram . . . 36
7.7 Use Case Diagram . . . 37
7.8 Iteration 3 . . . 38
7.8.1 System Architecture Diagram . . . 38
7.8.2 Flowchart Diagram . . . 39
7.8.3 Sequence Diagram . . . 40
7.8.4 Use Case Diagram . . . 41

8 System Implementation and Testing **42**
8.1 Technology Stack Implementation . . . 42
8.2 System Screenshots . . . 42
8.2.1 Dashboard Overview . . . 42
8.2.2 Indexing and Folder Management . . . 42
8.2.3 Semantic Search Results . . . 43
8.2.4 System Statistics Dashboard . . . 43
8.3 Implemented Features . . . 43
8.4 Implementation Statistics . . . 43
8.5 Code Quality and Best Practices . . . 44
8.6 Testing Methodology . . . 44
8.6.1 Frontend and API Interface Testing . . . 44
8.6.2 Search Engine and Re-Ranking Testing . . . 44
8.6.3 Data Extraction Pipeline Testing . . . 44
8.6.4 Regression Testing . . . 44
8.7 Implementation Achievements . . . 44

9 System Verification and Operational Deployment **45**
9.1 Continuous Integration and Testing . . . 45
9.2 System-Level Test Suites and Test Cases . . . 45
9.2.1 Test Case 1 – Multi-Modal File Indexing Accuracy . . . 45
9.2.2 Test Case 2 – Context-Preserving Chunking Validation . . . 45
9.2.3 Test Case 3 – Semantic Search Re-Ranking Precision . . . 46
9.2.4 Test Case 4 – Vector Database (FAISS) Persistence and Integrity 46
9.2.5 Test Case 5 – Anomaly and Malformed Data Handling . . . 46
9.2.6 Test Case 6 – Performance and Latency Metrics . . . 46
9.3 GitHub Repository: Lumina Semantic Search . . . 47

10 Implementation Details **48**
10.1 Technical Stack Overview . . . 48
10.2 System Architecture . . . 48
10.3 Integration Modules . . . 49
10.4 Environment Configuration . . . 49

11 User Manual **50**
11.1 Quick Start Guide 50
11.1.1 Step 1: Setup Environment 50
11.1.2 Step 2: Start AI Core (Optional) 50
11.1.3 Step 3: Run Application 50
11.2 Core Features 50
11.3 Troubleshooting 50

12 Conclusions and Future Work **51**
12.1 Project Summary 51
12.2 System Limitations 51
12.3 Future Work 51
12.4 Final Remarks 51

References **52**

List of Figures

6.1 Structural Design of Lumina System 20
6.2 Activity Diagram of Lumina System 21
6.3 Context Diagram of Lumina System 22
6.4 Entity-Relationship Diagram of Lumina 23
6.5 System Flow Diagram of Lumina 24
6.6 Sequence Diagram of Lumina System 25
6.7 Use Case Diagram of Lumina System 26

7.1 Structural Design of Lumina System (Iteration 2) 33
7.2 Activity Diagram of Lumina System (Iteration 2) 34
7.3 Call Graph of Lumina System 35
7.4 Entity-Relationship Diagram of Lumina (Iteration 2) 35
7.5 System Flow Diagram of Lumina (Iteration 2) 36
7.6 Sequence Diagram of Lumina System (Iteration 2) 37
7.7 Use Case Diagram of Lumina System (Iteration 2) 37
7.8 System Architecture Diagram of Lumina (Iteration 3) 39
7.9 Flowchart Diagram of Lumina System (Iteration 3) 40
7.10 Sequence Diagram of Lumina System (Iteration 3) 41
7.11 Use Case Diagram of Lumina System (Iteration 3) 41

Chapter 1

Introduction

1.1 Overview

The purpose of this investigation is to design and develop an intelligent, privacy-focused desktop search assistant named **Lumina**. The system performs multimodal search across textual, visual, and audio content entirely offline. It uses modern machine learning models to extract, understand, and retrieve information from user files stored locally on a computer.

The central problem being addressed is the inefficiency of existing search tools in handling diverse data formats such as scanned documents, recorded lectures, and multimedia content. Conventional search engines are text-based and cloud-dependent, raising both accessibility and privacy concerns. Lumina resolves this by offering a unified offline solution for semantic retrieval.

The background of this project lies in the rapid evolution of multimodal learning. Recent work in Optical Character Recognition (OCR), Automatic Speech Recognition (ASR), and embedding-based search—such as CLIP, MiniLM, and Whisper—has made it possible to represent multiple types of data in a shared semantic space. However, these solutions are either specialized for a single modality or require cloud computation. Lumina extends this by integrating models locally and combining them through a novel fusion algorithm.

The approach involves building a modular pipeline that extracts text from audio, images, and videos, generates embeddings, and stores them in a FAISS vector database for rapid retrieval. The models are optimized for multilingual support (English and Urdu) and for use on student-level hardware.

The criteria for success include:

- Accurate and efficient semantic search across documents, audio, and visual data
- Successful operation in offline environments
- Ability to handle Urdu and English datasets
- Clear visualization of results, including keyword heatmaps
- Modular architecture for future model improvements

1.2 Purpose of the Investigation

The investigation aims to develop a practical and intelligent tool that can help users, especially students, educators, doctors, and lawyers, retrieve meaningful information from large collections of mixed-format data. The project emphasizes accessibility, data privacy, and multimodal intelligence.

1.3 Problem Being Investigated

Users face challenges in locating precise information scattered across diverse digital formats. Traditional file search methods rely on filenames or basic text matching, ignoring the deeper context within multimedia content. The problem becomes more complex for Urdu and domain-specific data, where standard models lack accuracy.

1.4 Background and Importance

Prior research has demonstrated strong performance of models like Whisper for transcription and CLIP for image-text understanding. However, existing retrieval systems still lack offline capability and multilingual adaptability. Lumina integrates these models and introduces two custom algorithms—**Lumina-OCR Enhancer** and **Fusion Embedding Model**—to improve performance on local datasets.

1.5 Thesis and General Approach

The proposed solution employs multimodal AI components integrated through an Electron-based desktop interface. The backend, written in Python, processes user files using OCR and ASR, converts them into embeddings using fine-tuned models, and retrieves relevant segments through FAISS. The user interacts with an intuitive graphical interface to submit queries and visualize results.

1.6 Criteria for Success

Lumina will be considered successful if it can:

1. Accurately search and retrieve information across text, image, and audio files
2. Provide offline, privacy-preserving functionality
3. Support multilingual inputs and outputs (English and Urdu)
4. Display user-friendly visualizations such as keyword heatmaps
5. Operate efficiently on standard computing hardware

1.6.1 Project Scope

This prototype stage focuses on demonstrating the core functionalities—file upload, OCR/ASR processing, embedding generation, and semantic retrieval—laying the foundation for the complete version.

Demonstration

An early working prototype showcases the flow from file ingestion to retrieval. Future iterations will integrate custom algorithms and real-world datasets for enhanced performance.

Chapter 2

Review of Literature

A comprehensive review was conducted to evaluate recent work in multimodal retrieval systems, OCR enhancement, and model fine-tuning for low-resource languages such as Urdu. The review focuses on identifying existing methodologies, assessing their performance, and highlighting research gaps relevant to the Lumina project.

2.1 Introduction

The literature review explores multimodal information retrieval frameworks, including model architectures that combine textual and visual embeddings. It also considers OCR and ASR advancements, with attention to fine-tuning strategies for local languages.

2.2 Existing Work

- Studies such as *Enhancing Multimodal Retrieval via Complementary Information Extraction (2025)* and *Joint Fusion and Encoding (2025)* propose architectures for integrating visual and textual data.
- Research on *Whisper Fine-Tuning for Low-Resource Languages (2024)* provides methods to adapt speech recognition systems for Urdu.
- OCR-focused works like *PreP-OCR (2025)* present restoration and post-correction techniques relevant to the Lumina-OCR Enhancer.
- Surveys on *Multimodal Retrieval-Augmented Generation (2025)* discuss the integration of retrieval with generative models, aligning with Lumina's summarization goals.

2.3 Tabular Analysis of Literature

2.4 Conclusion

The literature establishes that while significant progress has been made in multimodal retrieval, offline capability and low-resource language adaptation remain underdeveloped. Lumina addresses these gaps by combining fine-tuned and custom-built models for local domains, providing a strong foundation for offline intelligent search.

Table 2.1: Summary of Selected Research Papers

Paper Title	**Key Contribution**	**Relevance to Lumina**	**Year**
Enhancing Multimodal Retrieval via Complementary Information Extraction	Joint use of textual and visual embeddings	Core reference for fusion techniques	2025
PreP-OCR: Document Restoration and Correction	OCR quality improvement for degraded scans	Basis for Lumina-OCR Enhancer	2025
Whisper Fine-Tuning for Urdu	Adapting ASR to low-resource languages	Urdu speech integration	2024
FAISS: Library for Efficient Similarity Search	High-speed vector retrieval	Search backbone	2024

Chapter 3

Project Vision

3.1 Problem Statement

Modern users work with documents, images, and media scattered across multiple devices, making information retrieval both time-consuming and inefficient. Existing tools often fail to provide unified access to such diverse data formats. **Lumina** addresses this issue by offering an integrated offline search system capable of understanding and indexing multimodal data.

3.2 Business Opportunity

There is an increasing demand for privacy-preserving intelligent systems, particularly in sectors such as academia and healthcare. Lumina presents a valuable solution for universities, hospitals, and law firms that require secure internal data retrieval without reliance on cloud-based services.

3.3 Objectives

The primary objectives of the Lumina project are:

- Develop an offline multimodal search assistant.
- Implement OCR and ASR systems fine-tuned for both Urdu and English.
- Introduce the Lumina-OCR Enhancer and Fusion Embedding algorithms.
- Enable cross-modal retrieval and visualization of data.

3.4 Project Scope

The system focuses on four major file categories: PDFs, images, audio, and video. Each file type is processed, indexed, and made searchable within a local environment. The initial phase demonstrates system functionality using smaller datasets.

3.5 Constraints

The project is subject to the following limitations:

- Limited GPU memory for model execution.
- Scarcity of high-quality Urdu datasets for fine-tuning.
- Offline environment restricting access to cloud-scale computational resources.

3.6 Stakeholders Description

3.6.1 Stakeholders Summary

- **Students:** Use Lumina to search lecture slides, notes, and recorded sessions.
- **Professors:** Organize and retrieve academic materials efficiently.
- **Doctors:** Access patient reports and voice dictations locally.
- **Lawyers:** Quickly locate relevant legal documents and case files.

3.6.2 Key High-Level Goals and Problems of Stakeholders

- Reduce the time required to search through large volumes of files.
- Improve accessibility of both Urdu and English data.
- Ensure data confidentiality by maintaining a fully offline system.

Chapter 4

Software Requirements Specification

4.1 Introduction

4.1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements of Lumina, an offline, multimodal search assistant that can understand and retrieve information across text, image, audio, and video formats.

The purpose of this document is to describe the scope, features, behavior, and performance characteristics of the system for developers, supervisors, and evaluators. It serves as a guide throughout design, implementation, and testing.

4.1.2 Document Conventions

- REQ-# denotes a unique requirement identifier.
- Priorities are classified as High (H), Medium (M), or Low (L).
- Diagrams follow UML conventions (Use Case, Sequence, DFD).
- Technical terms are defined in Appendix A.
- All requirements are verifiable and traceable.

4.1.3 Intended Audience and Reading Suggestions

This document is intended for:

- Supervisors / Jury Members
- Developers
- Testers
  - End Users 0) professors doctors lawyers)

Readers may begin with Section 2 for an overview and proceed to Section 4 for detailed requirements.

4.1.4 Product Scope

Lumina is a privacy-focused desktop application that allows users to perform intelligent, semantic search across multiple file types—documents (PDF, DOCX), images, audio, and videos—completely offline.

It combines automatic speech recognition, optical character recognition, image–text embedding, and large language models.

Key Goals:

- Offline multimodal search
- Urdu + English support
- Lumina-OCR Enhancer
- Fusion Embedding Model
- Keyword heatmap visualization

4.1.5 References

- IEEE Std 830-1998
- FAISS Documentation (Facebook AI Research, 2024)
- Whisper, CLIP, MiniLM repositories

4.2 Overall Description

4.2.1 Product Perspective

Lumina is a stand-alone system with a hybrid architecture:

- Frontend: Electron.js
- Backend: Python
- Storage: FAISS vector database

4.2.2 Product Functions

1. Upload and manage files
2. Perform OCR and ASR
3. Generate embeddings
4. Store and retrieve via FAISS
5. Process queries
6. Summarize results
7. Display heatmaps

4.2.3 User Classes and Characteristics

User Class	Description	Technical Level
Students	Notes, lectures, assignments	Basic
Professors/Researchers	Research data	Intermediate
Doctors	Patient records	Intermediate
Lawyers	Legal documents	Intermediate
Developers	Manage system	Advanced

4.2.4 Operating Environment

- Platform: Windows
- Frontend: Electron
- Backend: Python
- Dependencies: PyTorch, OpenCV, FAISS, Tesseract, Whisper, CLIP, MiniLM

Hardware Requirements:

- Minimum: 8GB RAM
- Recommended: GPU support

4.2.5 Design and Implementation Constraints

- Must operate offline
- Open-source models only
- Urdu + English support
- Heavy tasks may use Colab
- Follow IEEE standards

4.2.6 User Documentation

- User Manual
- Installation Guide
- API Documentation
- Tutorial + Demo

4.2.7 Assumptions and Dependencies

- Supported file formats provided
- GPU or Colab available
- Model accuracy depends on fine-tuning
- Internet needed only for updates

4.3 External Interface Requirements

4.3.1 User Interfaces

- Dashboard
- Visualization (heatmap)
- Settings panel

4.3.2 Hardware Interfaces

- Microphone
- Local storage
- GPU

4.3.3 Software Interfaces

- Whisper (ASR)
- Tesseract (OCR)
- CLIP + MiniLM
- FAISS
- SQLite

4.3.4 Communication Interfaces

- REST API (Electron Python)
- Optional Colab integration
- No external data sharing

4.4 System Features

4.4.1 Feature 1: Multimodal Search Engine

Priority: High

- REQ-1: Convert media to text
- REQ-2: Generate embeddings
- REQ-3: Urdu + English support
- REQ-4: Semantic retrieval

4.4.2 Feature 2: Lumina-OCR Enhancer

Priority: High

- REQ-5: Preprocessing
- REQ-6: OCR execution
- REQ-7: Post-correction

4.4.3 Feature 3: Fusion Embedding Model

Priority: High

- REQ-8: Compute embeddings
- REQ-9: Fusion
- REQ-10: Store in FAISS

4.4.4 Feature 4: Keyword Heatmap

Priority: Medium

- REQ-11: Keyword density
- REQ-12: Heatmap rendering
- REQ-13: Highlight peaks

4.4.5 Feature 5: Summarization

Priority: Medium

- REQ-14: Generate summaries
- REQ-15: Provide answers

4.5 Other Nonfunctional Requirements

4.5.1 Performance

- Response < 2 seconds
- OCR accuracy 90%
- ASR WER 15%
- Heatmap < 1 second

4.5.2 Safety

- No file modification
- Safe logging

4.5.3 Security

- Fully offline
- Encrypted storage
- Restricted access

4.5.4 Software Quality Attributes

- Usability
- Portability
- Maintainability
- Reliability
- Extensibility

4.5.5 Business Rules

- Verified users only
- Urdu-English consistency

4.6 Other Requirements

- Logging for evaluation
- Model performance reports
- Ethical compliance

4.7 Appendices

4.7.1 Appendix A: Glossary

Term	Definition
ASR	Automatic Speech Recognition
OCR	Optical Character Recognition
LLM	Large Language Model
FAISS	Similarity Search Library
Embedding	Vector representation of data
Heatmap	Visual keyword intensity map

4.7.2 Appendix B: Analysis Models

- Use Case: Upload → Process → Search → Visualize
- DFD: Input → Processing → Storage → Output

4.7.3 Appendix C: To Be Determined

- Urdu dataset sources
- Fine-tuning parameters
- UI design

Chapter 5

Iteration Plan

The development of **Lumina – Deep Search Assistant Across All Extensions** follows an iterative and incremental approach aligned with the two-semester Final Year Project structure at NUCES FAST – Peshawar.

The project is divided into four major evaluation milestones:

- Midterm FYP-1
- Final FYP-1
- Midterm FYP-2
- Final FYP-2

Each phase builds on the previous one, ensuring continuous improvement, risk reduction, and steady progress toward a fully functional multimodal desktop search assistant.

5.1 Midterm FYP-1 (August 2025 – October 2025)

Focus: Requirement analysis, planning, and system design

Key Deliverables

- Finalized project title and supervisor allocation
- Approved Project Proposal (08 August 2025)
- Software Requirements Specification (SRS v1.0)
- Complete System Design Documentation, including:
  - Use Case Diagram
  - Data Flow Diagrams (Level 0 & 1)
  - Sequence & Activity Diagrams
  - Entity–Relationship Diagram (Lumina Metadata Model)
  - Component Diagram & High-Level Architecture Diagram
  - Data Structure Design

- Initial UI wireframes and Figma mockups
- Literature review on:
  - Multimodal embeddings
  - FAISS similarity search
  - Offline AI and local model deployment

Outcome

A fully approved analysis and design foundation, ready for implementation.

5.2 Final FYP-1 (November 2025 – January 2026)

Focus: Backend implementation and metadata management

Key Deliverables

- Fully functional SQLite metadata database with refined ER diagram
- File ingestion pipeline for:
  - PDF, DOCX, images, audio, video
- Chunking module (text pages, audio timestamps, video keyframes)
- Embedding generation using:
  - MiniLM
  - CLIP
  - Whisper-base
- FAISS vector index with `faiss_ref` mapping
- Hybrid keyword + semantic search prototype
- Initial Electron + React desktop shell with drag-and-drop
- FYP-1 Report (design + partial implementation)

Outcome

A working offline search prototype capable of indexing a folder and performing basic semantic search.

5.3 Midterm FYP-2 (February 2026 – April 2026)

Focus: Advanced features, fine-tuning, and UI enhancement

Key Deliverables

- Fusion embedding model (late fusion of text + image + audio)
- Whisper fine-tuning for Urdu-English code-mixed audio (LoRA)
- CLIP fine-tuning for lecture slides and handwritten notes
- Multimodal queries (search via text, voice, image)
- Knowledge timeline with timestamps and keyword heatmaps
- Chat-style interface with local LLM summarization and Q/A
- Performance optimizations:
  - GPU acceleration
  - Batch ingestion
  - IVF indexing
- Full testing suite (unit, integration, usability)

Outcome

A nearly complete system with intelligent search, improved UI, and optimized performance.

5.4 Final FYP-2 (May 2026 – July 2026)

Focus: Deployment, documentation, security, and final polishing

Key Deliverables

- Cross-platform installers:
  - Windows (.exe)
  - Linux (AppImage) – Future goal (subject to final decision)
- End-to-end encryption for database and FAISS index
- User management with role-based access:
  - Student
  - Teacher
  - Doctor
  - Lawyer
- Query logging and feedback mechanism
- Complete User Manual and official Video Demo

- Final FYP-2 report (results, evaluation, metrics, future work)
- Source code submission with documentation and licensing
- Final presentation and live demonstration

Chapter 6

Iteration 1

The first iteration corresponds to the Midterm of FYP-1 and focuses on producing the initial system design artifacts.

While requirements analysis remains consistent across the entire system, this iteration introduces the core design models, divided into two major categories:

- Structural Design
- Behavioral Design

These design artifacts form the foundation for system understanding, architecture planning, and subsequent implementation phases.

6.1 Structural Design

The structural design describes how the system is organized, including its components, layers, classes, and data structures. It defines the static architecture of **Lumina**.

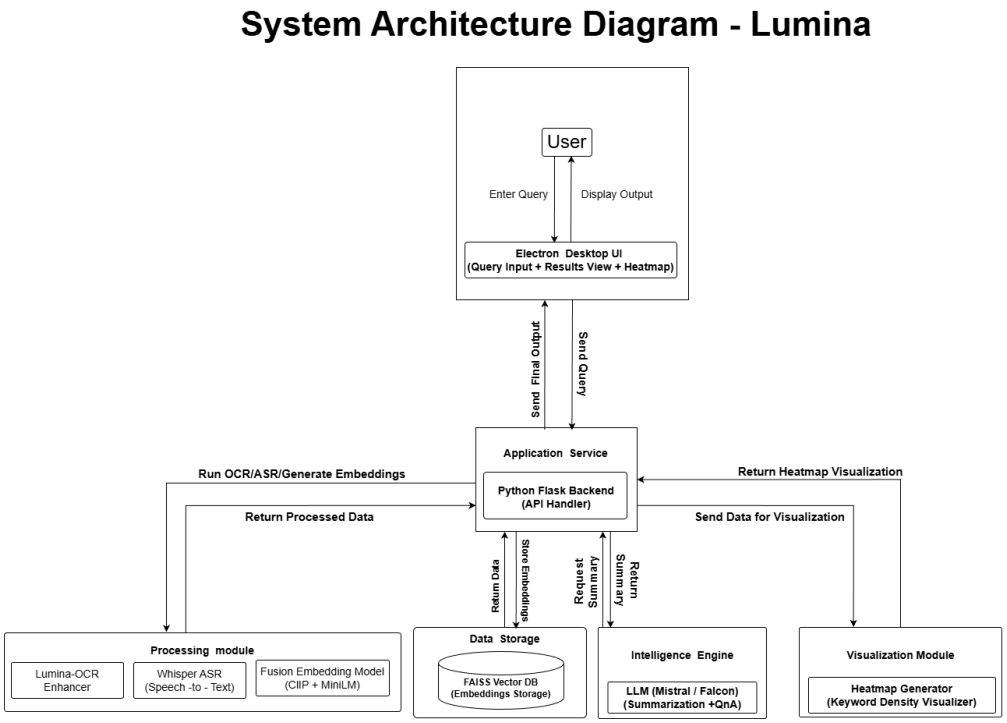


Figure 6.1: Structural Design of Lumina System

6.2 Activity Diagram

The activity diagram depicts the workflow of user and system interactions within Lumina. Key activities include:

- Selecting a folder
- Indexing files
- Performing queries
- Viewing results

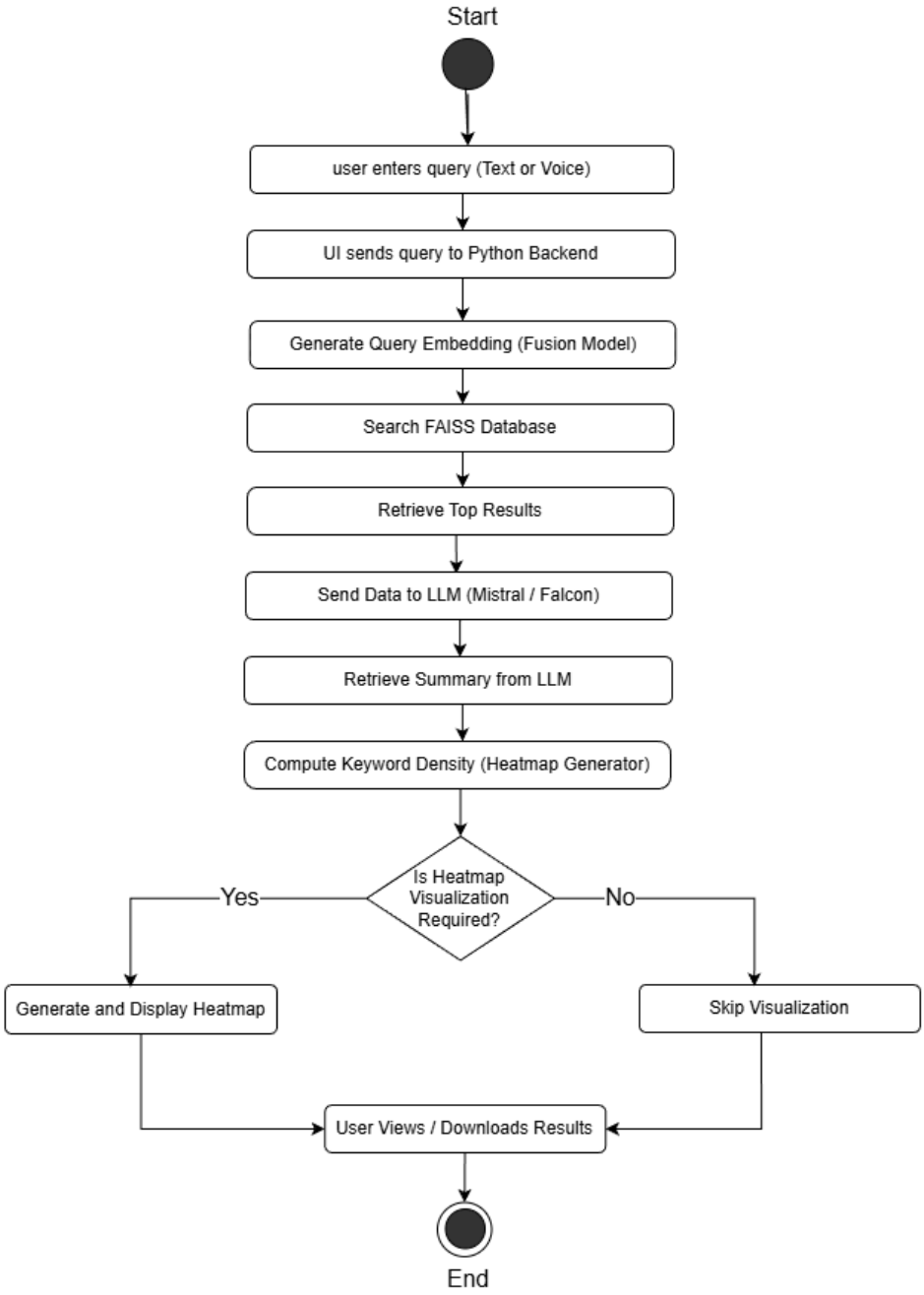


Figure 6.2: Activity Diagram of Lumina System

6.3 Context Diagram

The context diagram represents the overall system boundary and its interaction with external entities such as users and data sources.

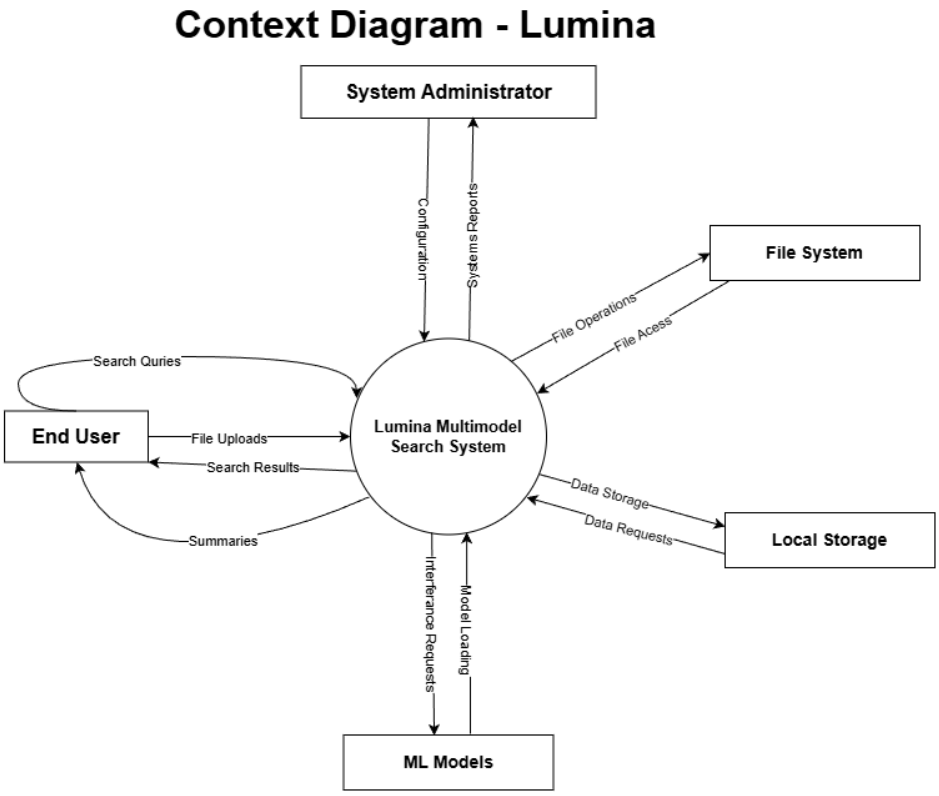


Figure 6.3: Context Diagram of Lumina System

6.4 ER Diagram

The Entity-Relationship Diagram (ERD) represents the database structure of Lumina, including metadata storage, embeddings, FAISS references, file records, and user management.



Figure 6.4: Entity-Relationship Diagram of Lumina

6.5 Flow Diagram

The flow diagram illustrates the complete process flow, from file ingestion to embedding generation and semantic search.

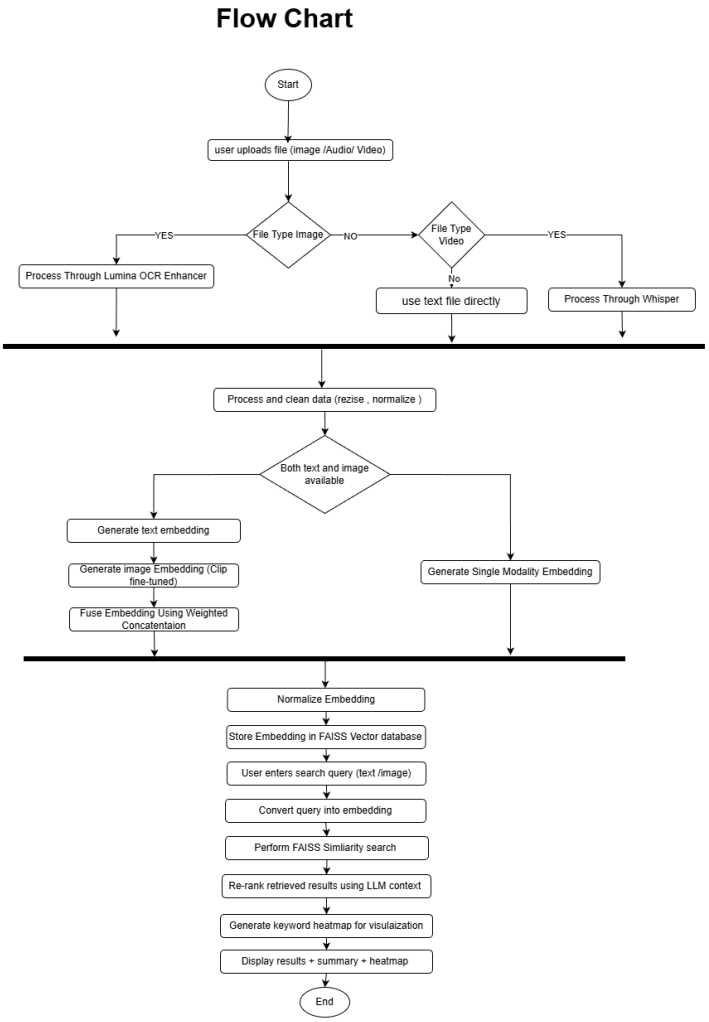


Figure 6.5: System Flow Diagram of Lumina

6.6 Sequence Diagram

The sequence diagram shows the interaction between system components over time, including user input, backend processing, and result retrieval.

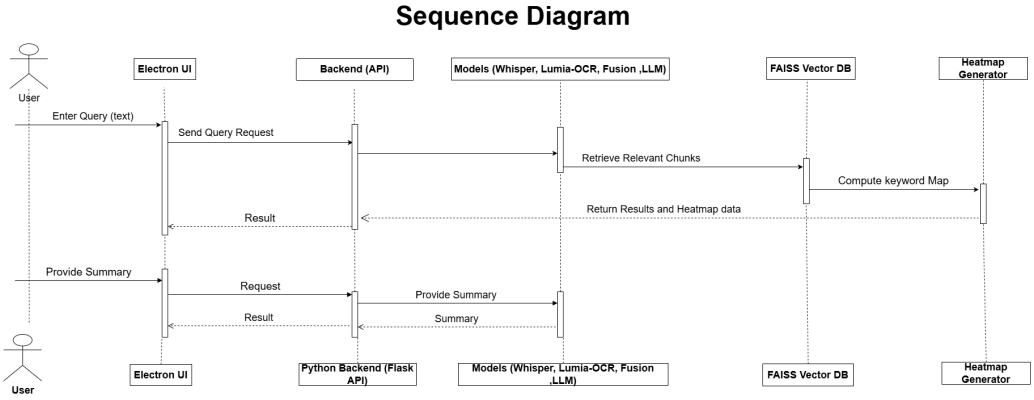


Figure 6.6: Sequence Diagram of Lumina System

6.7 Use Case Diagram

The use case diagram illustrates the interactions between users and the Lumina system, highlighting key functionalities such as file upload, indexing, search, and visualization.

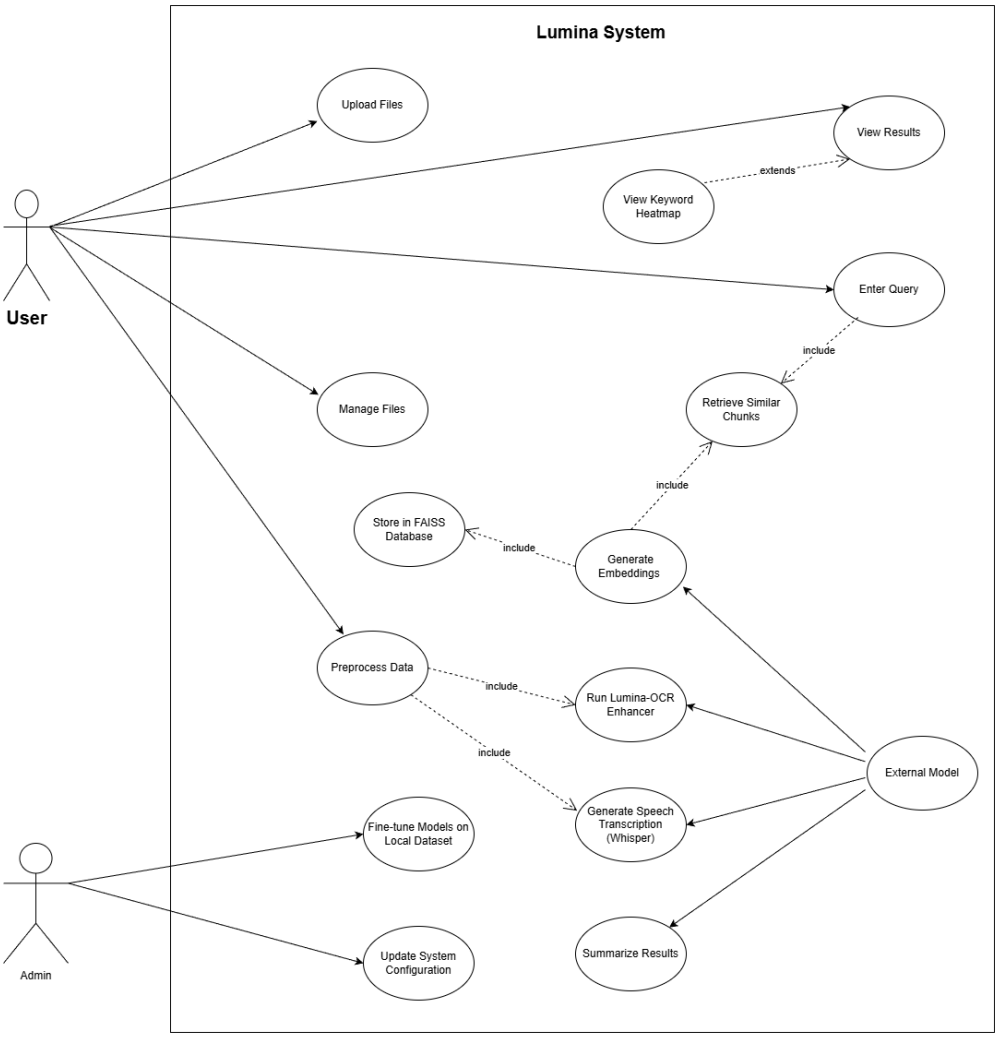


Figure 6.7: Use Case Diagram of Lumina System

6.8 Data Structure Design

Lumina employs efficient and lightweight data structures to support multimodal semantic search on consumer-grade hardware while ensuring complete offline privacy. The design focuses on three key aspects: storage of embeddings, representation of content chunks, and ranking of search results.

6.8.1 Storage of Embeddings (Vectors)

Embeddings generated from text (MiniLM), images and video frames (CLIP), and audio transcripts (Whisper/Fusion model) are high-dimensional dense vectors ranging from 384 to 768 dimensions (`float32`). Storing these vectors directly in a relational database would introduce significant performance overhead.

To address this, embeddings are stored in a FAISS (Facebook AI Similarity Search) vector index using `IndexFlatIP` (inner product) for smaller datasets and `IndexIVFFlat`

for larger collections. This enables efficient approximate nearest-neighbor search with sub-second retrieval times, even for thousands of data chunks.

Example (Python Backend):

```
import faiss
import numpy as np

dimension = 384
index = faiss.IndexFlatIP(dimension)

vectors = np.array(embeddings).astype('float32')
index.add(vectors)
```

The relational table `embeddings` stores only an integer reference, `faiss_ref`, which maps each chunk to its corresponding vector position in the FAISS index.

6.8.2 Representation of Chunks (List of Dictionaries)

Each file is divided into logical chunks, such as pages for PDFs, fixed-duration segments for audio/video, and keyframes for videos. During processing, these chunks are represented using Python dictionaries to ensure flexibility, readability, and ease of manipulation.

Example:

```
chunk = {
    "chunk_id": 1,
    "file_id": "lecture1.pdf",
    "text": "Introduction to AI",
    "timestamp": None,
    "page": 1
}
```

All chunks from a file are stored in a list structure:

```
chunks_list = [chunk1, chunk2, ..., chunkN]
```

6.8.3 Ranking of Search Results (Priority Queue / Heap)

FAISS returns similarity scores for the top-k candidate results. To ensure globally optimal ranking and support hybrid scoring (semantic + keyword-based), the results are further processed using Python's `heapq` module as a max-heap.

Example:

```
import heapq

results = []
for score, chunk in candidates:
    heapq.heappush(results, (-score, chunk))

top_k = [heapq.heappop(results) for _ in range(k)]
```

6.9 Algorithm Design

6.9.1 Lumina-OCR Enhancer

Goal: Improve OCR quality on scanned or low-quality documents and Urdu text by combining image restoration, region-aware OCR, and semantic post-correction.

High-level Description:

Lumina-OCR Enhancer improves image quality through denoising, contrast enhancement, and deskewing. It then detects text regions and layout (paragraphs, headers, columns), applies OCR on cleaned regions, and performs semantic post-correction using language models. The system is optimized for Urdu and local document layouts and returns structured text with positional metadata and confidence scores.

Evaluation Plan

- **Baseline:** Standard Tesseract OCR (without enhancement)
- **Dataset:** Mixed dataset (clean + degraded, Urdu + English)
- **Metrics:** CER (Character Error Rate), WER (Word Error Rate)
- **Procedure:**
    1. Run baseline OCR and compute CER/WER
    2. Run Lumina-OCR Enhancer and compute CER/WER
    3. Compare improvements (absolute and relative)
    4. Provide qualitative before/after comparisons
- **Statistical Test:** Paired Wilcoxon test or paired t-test

Pseudocode

```
Algorithm: Lumina-OCR Enhancer
Input: Document (image/PDF)
Output: Corrected text with metadata

1. START
2. Load document
3. For each page:
    a. Convert to RGB and normalize
    b. Preprocess:
        - Denoise
        - Enhance contrast
        - Deskew
        - Binarize
    c. Detect layout (EAST/CRAFT)
    d. Extract text regions
    e. Run OCR on each region
    f. Collect text + confidence
    g. Post-process text
```

```
    h. Apply semantic correction
    i. Re-run OCR if confidence low
4. Merge results
5. Attach metadata
6. Export structured output
7. END
```

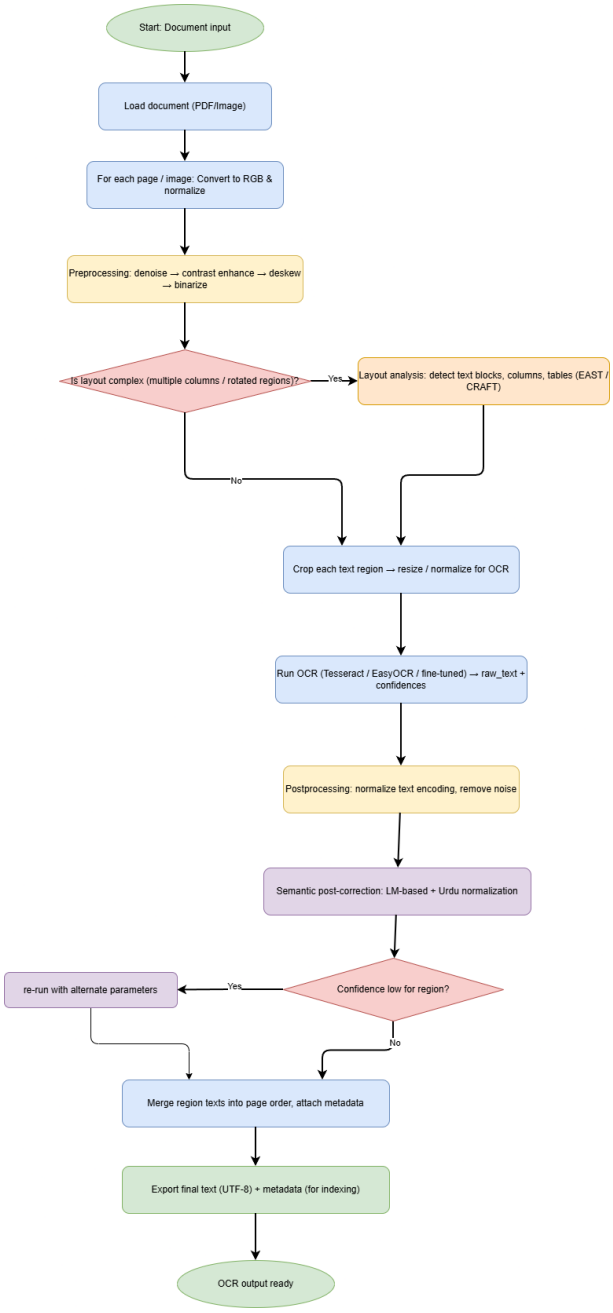


Figure 6.8: Lumina-OCR Enhancer Pipeline

6.9.2 Fusion Embedding Model

Goal: Generate a unified multimodal vector combining textual and visual information for efficient retrieval.

High-level Description:

For each content chunk, Lumina computes text embeddings (MiniLM) and image embeddings (CLIP). These are fused using weighted averaging or concatenation to form a single vector stored in FAISS. At query time, embeddings are generated dynamically based on input modality and used for similarity search.

Design Considerations

- Lightweight design suitable for offline environments
- Fusion tuned using validation metrics (Precision@k, MRR)
- Optional projection layer trained using contrastive learning
- Metadata stored for visualization and navigation

Pseudocode

```
Algorithm: Fusion Embedding Model
Input: Query (text/image)
Output: Ranked results

1. START
2. Receive query
3. Preprocess input:
    - Clean text
    - Normalize image
4. Generate embeddings:
    - Text → MiniLM
    - Image → CLIP
5. If both exist:
    - Fuse embeddings
6. Else:
    - Use single embedding
7. Normalize vector
8. Store in FAISS
9. On search:
    a. Compute query embedding
    b. Retrieve top-k results
    c. Re-rank using LLM
    d. Generate summaries
    e. Compute heatmaps
10. Display results
11. END
```

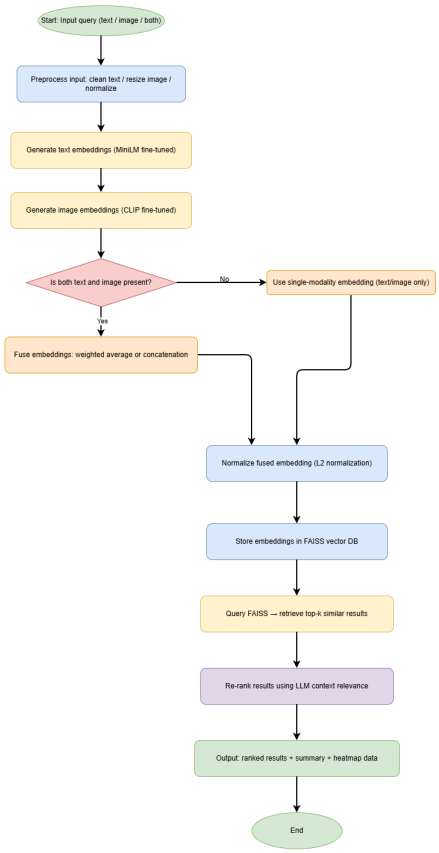


Figure 6.9: Fusion Embedding Model Workflow

Chapter 7

Iteration 2

The second iteration corresponds to the final phase of FYP-1 development. This iteration focuses on refining the system design and preparing it for full implementation. Improved architectural models, refined workflows, and updated data structures were developed based on insights gained during the first iteration.

Similar to Iteration 1, the design artifacts are divided into two major categories:

- Structural Design
- Behavioral Design

These artifacts represent the evolved architecture of the Lumina system and provide a clearer representation of the implementation pipeline.

7.1 Structural Design

The structural design illustrates the improved architecture of the Lumina system, including internal modules, AI processing components, and the interaction between indexing, embedding, and search engines.

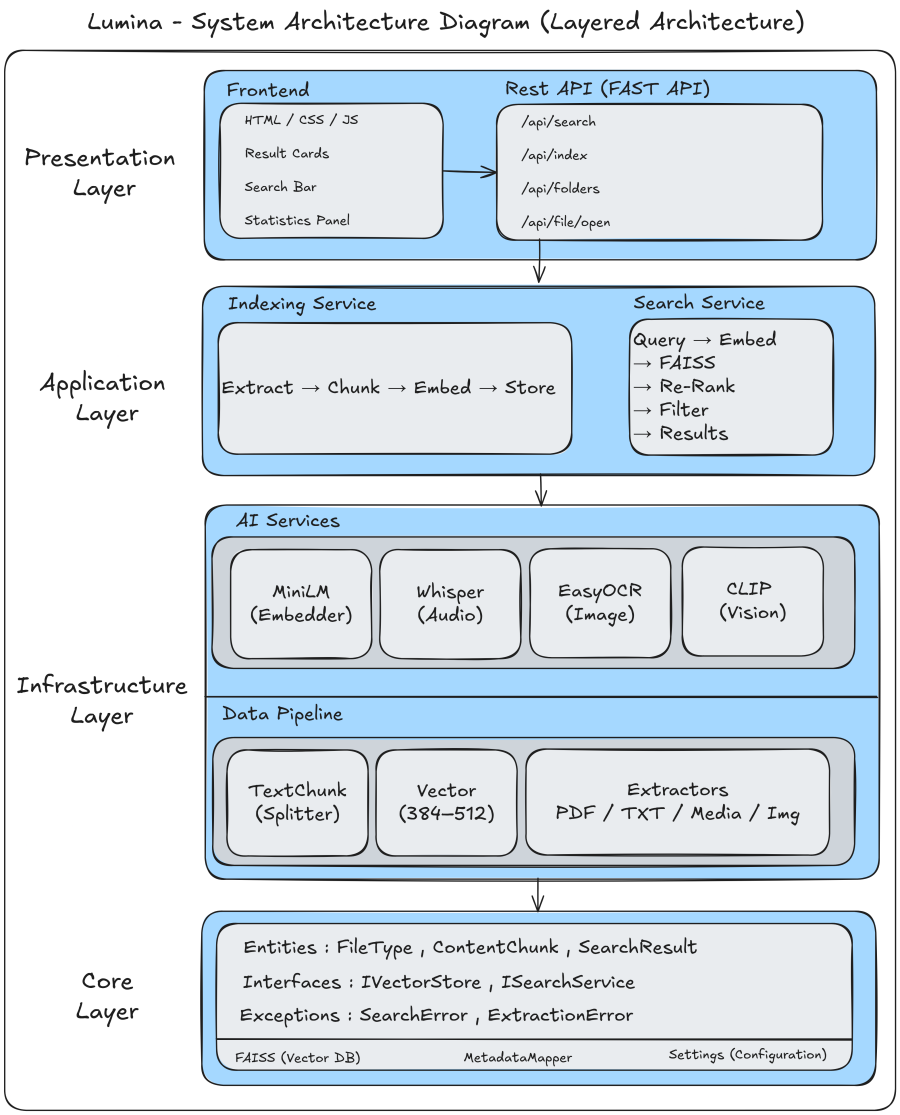


Figure 7.1: Structural Design of Lumina System (Iteration 2)

7.2 Activity Diagram

The activity diagram represents the refined workflow of the Lumina system, including indexing, processing of multimodal data, and semantic search execution.

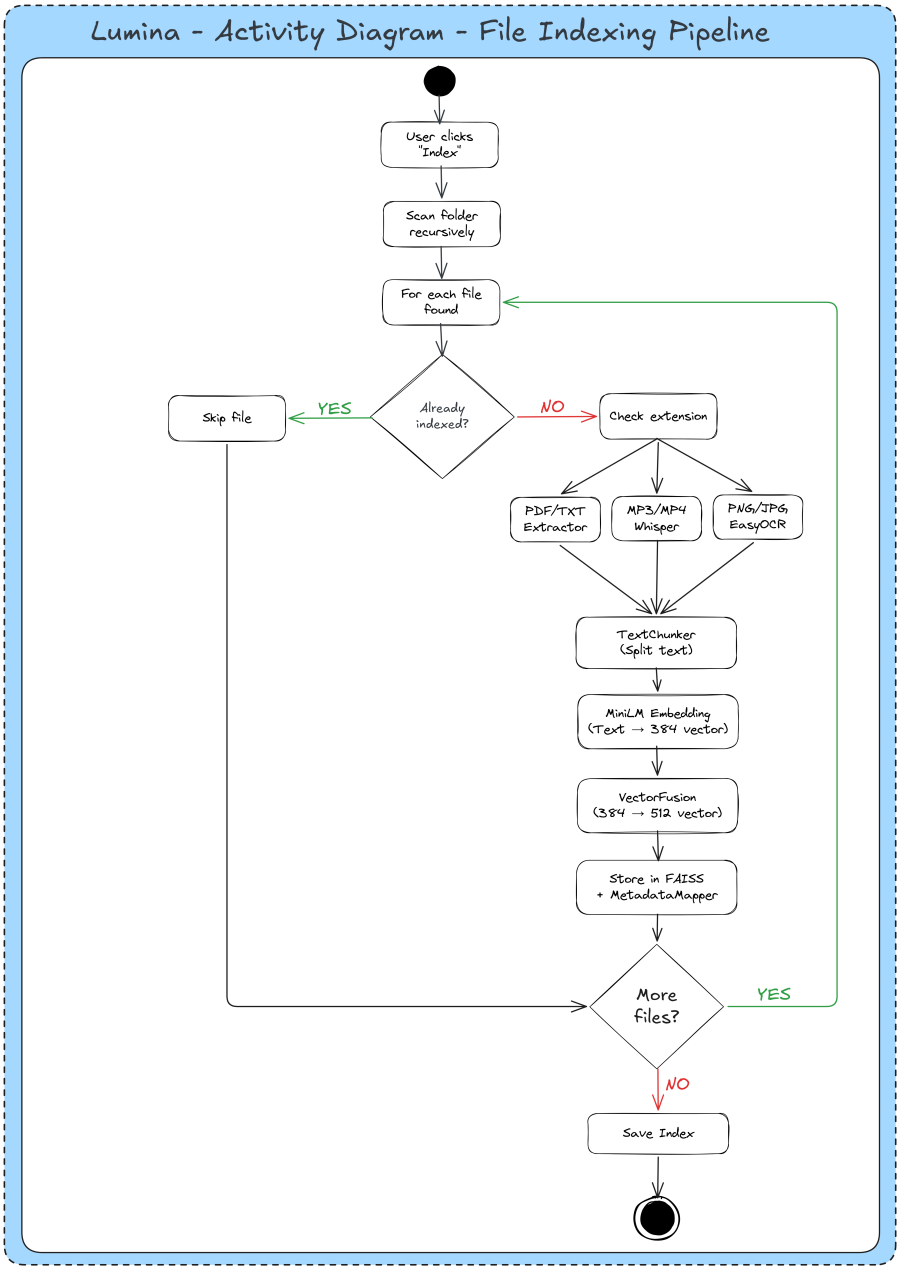


Figure 7.2: Activity Diagram of Lumina System (Iteration 2)

7.3 Call Graph

The call graph illustrates how different internal functions and modules interact during execution. It highlights the flow of function calls from user interaction through the backend processing pipeline.

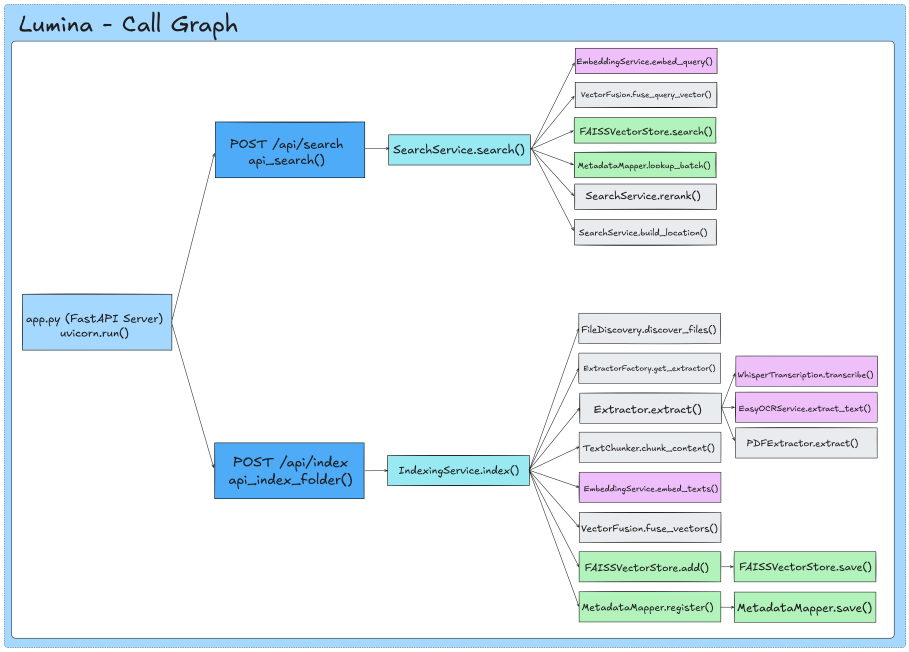


Figure 7.3: Call Graph of Lumina System

7.4 ER Diagram

The updated Entity-Relationship Diagram represents the refined database structure used for managing system metadata, indexed files, embeddings, and configuration records.

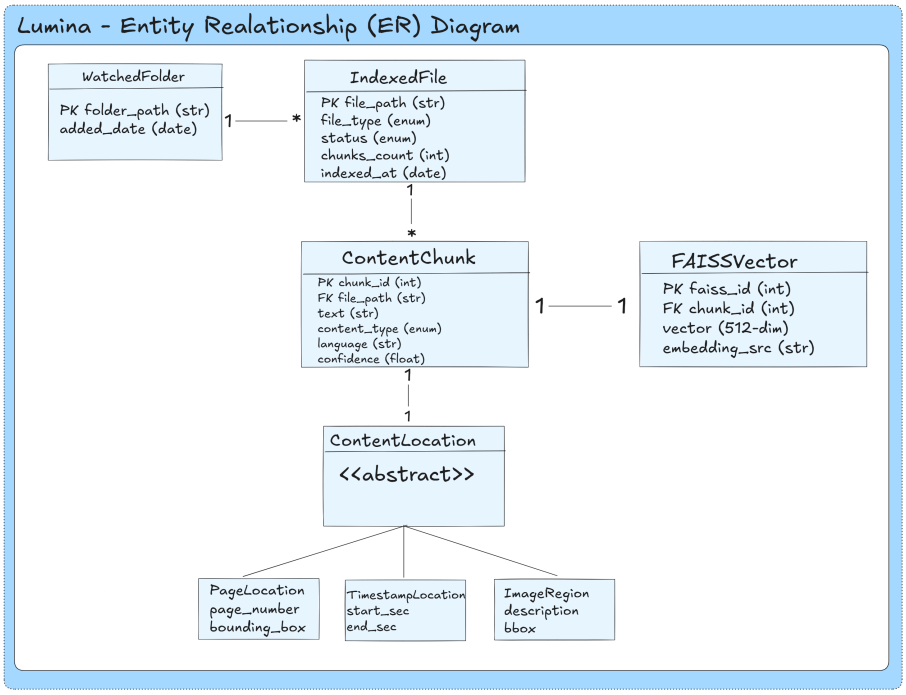


Figure 7.4: Entity-Relationship Diagram of Lumina (Iteration 2)

7.5 Flow Diagram

The flow diagram illustrates the optimized data processing pipeline, starting from file ingestion to embedding generation and semantic search execution.

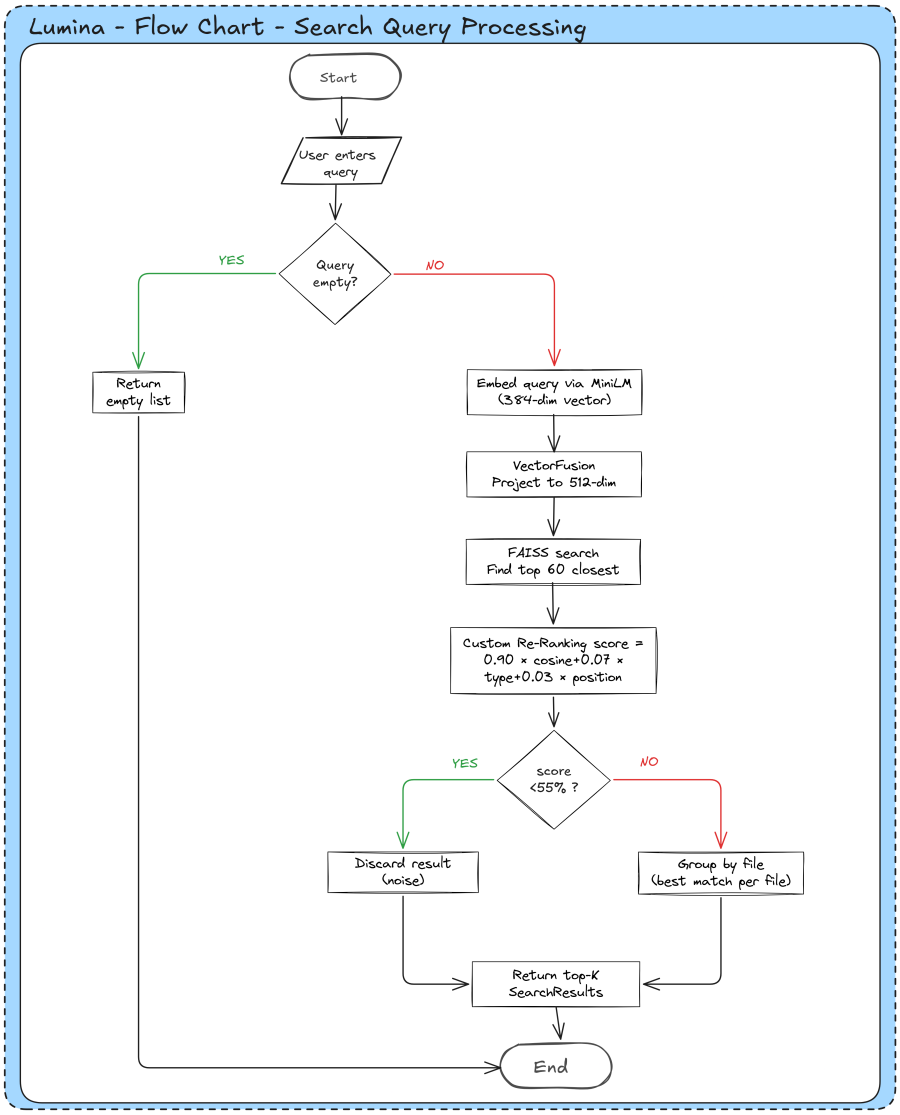


Figure 7.5: System Flow Diagram of Lumina (Iteration 2)

7.6 Sequence Diagram

The sequence diagram demonstrates the time-based interactions between the user interface, backend services, AI modules, and the vector database during query processing.

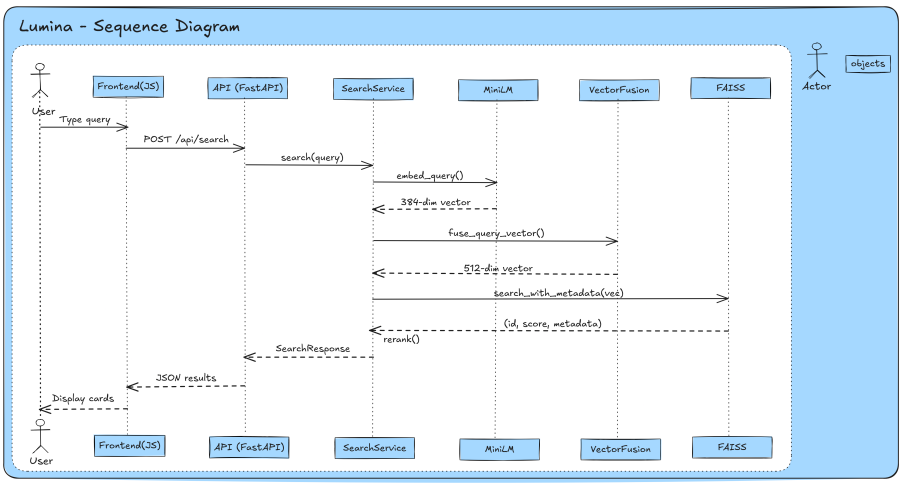


Figure 7.6: Sequence Diagram of Lumina System (Iteration 2)

7.7 Use Case Diagram

The updated use case diagram highlights the primary functionalities available to users including folder indexing, semantic search, file retrieval, and analytics visualization.

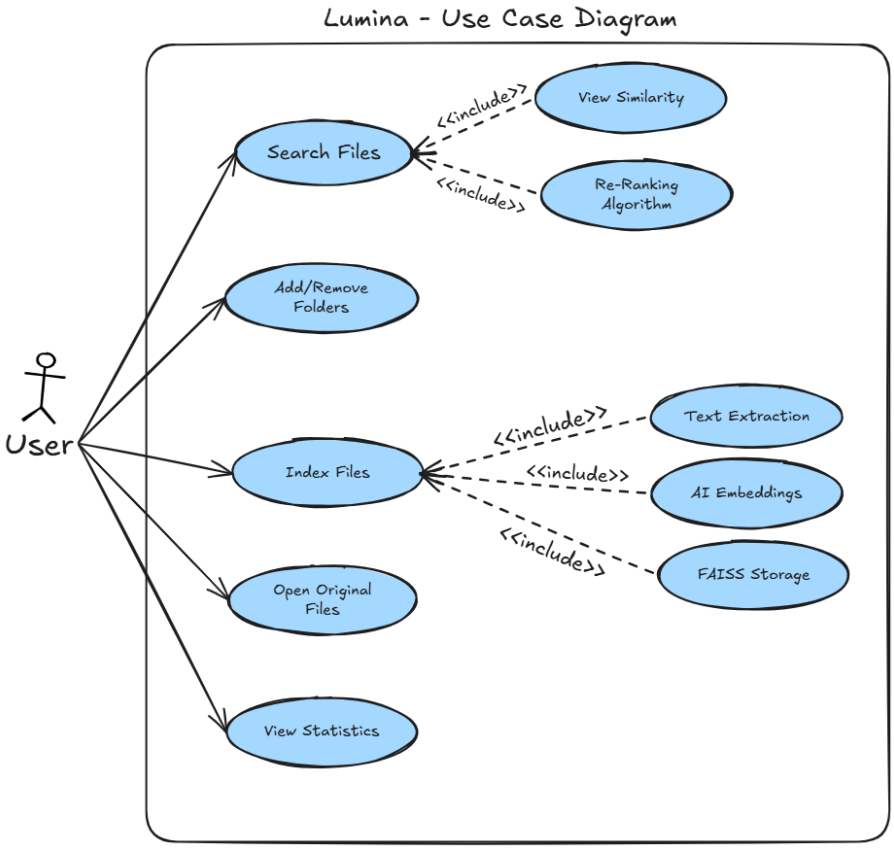


Figure 7.7: Use Case Diagram of Lumina System (Iteration 2)

7.8 Iteration 3

The third iteration represents the final refinement and completion of the Lumina system design. This iteration focuses on enhancing multimodal capabilities, improving the unified vector space, implementing advanced re-ranking, and optimizing the overall search pipeline for better accuracy and performance.

Based on feedback and insights from Iteration 2, significant improvements were made to the indexing workflow, query processing, and system architecture. The following diagrams illustrate the matured and final design of the Lumina semantic search engine.

7.8.1 System Architecture Diagram

The system architecture diagram presents the final layered architecture of Lumina, clearly separating the Presentation, Application, AI Services, Data Pipeline, and Core layers. It highlights the integration of multiple AI models (MiniLM, Whisper, EasyOCR, and CLIP) and the unified vector processing pipeline.

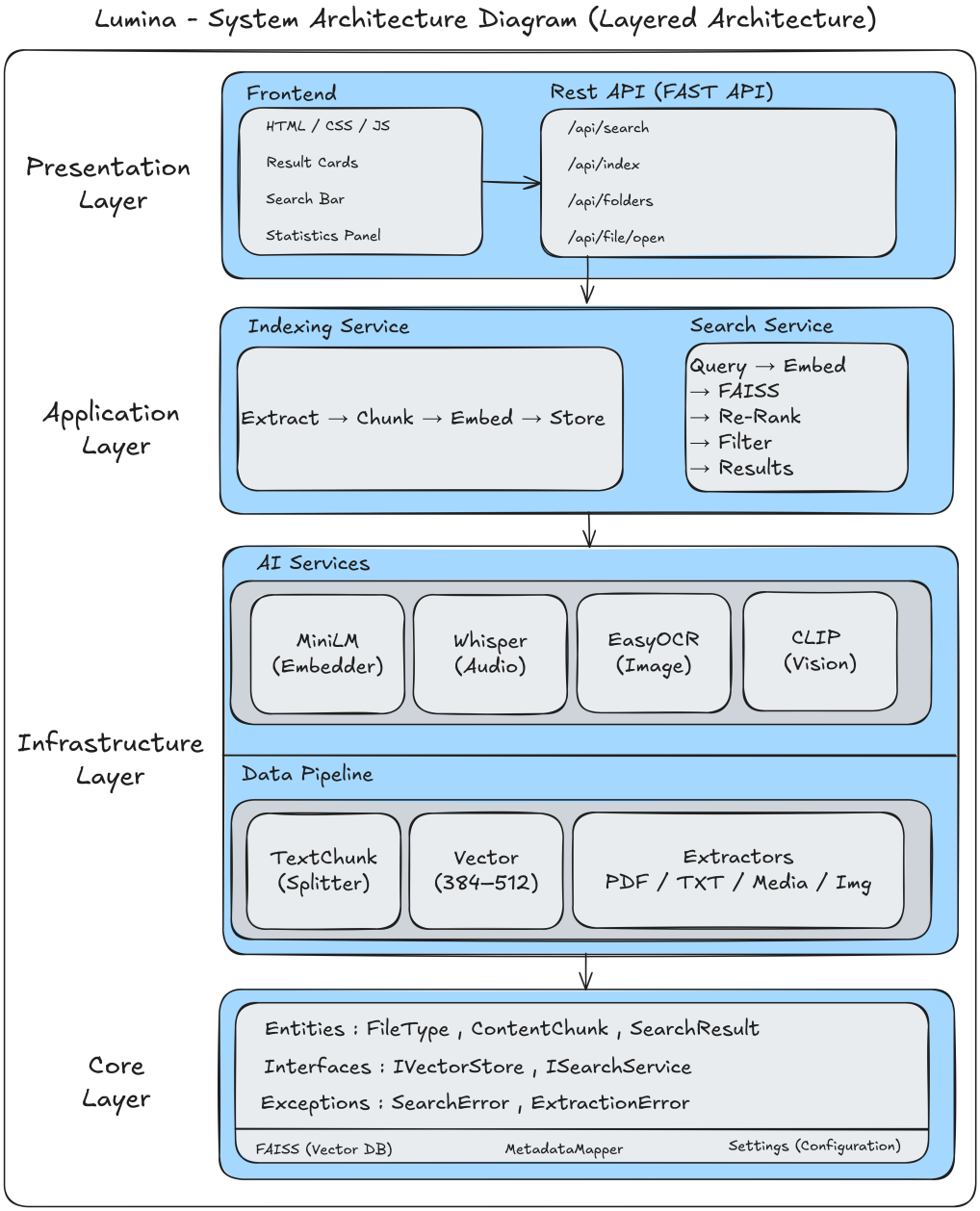


Figure 7.8: System Architecture Diagram of Lumina (Iteration 3)

7.8.2 Flowchart Diagram

The flowchart depicts the complete query processing pipeline in Lumina. It covers query embedding, vector fusion, FAISS search, custom re-ranking with dynamic thresholding, smart collection filtering, result grouping, and optional RAG-based AI summary generation using a local LLM.

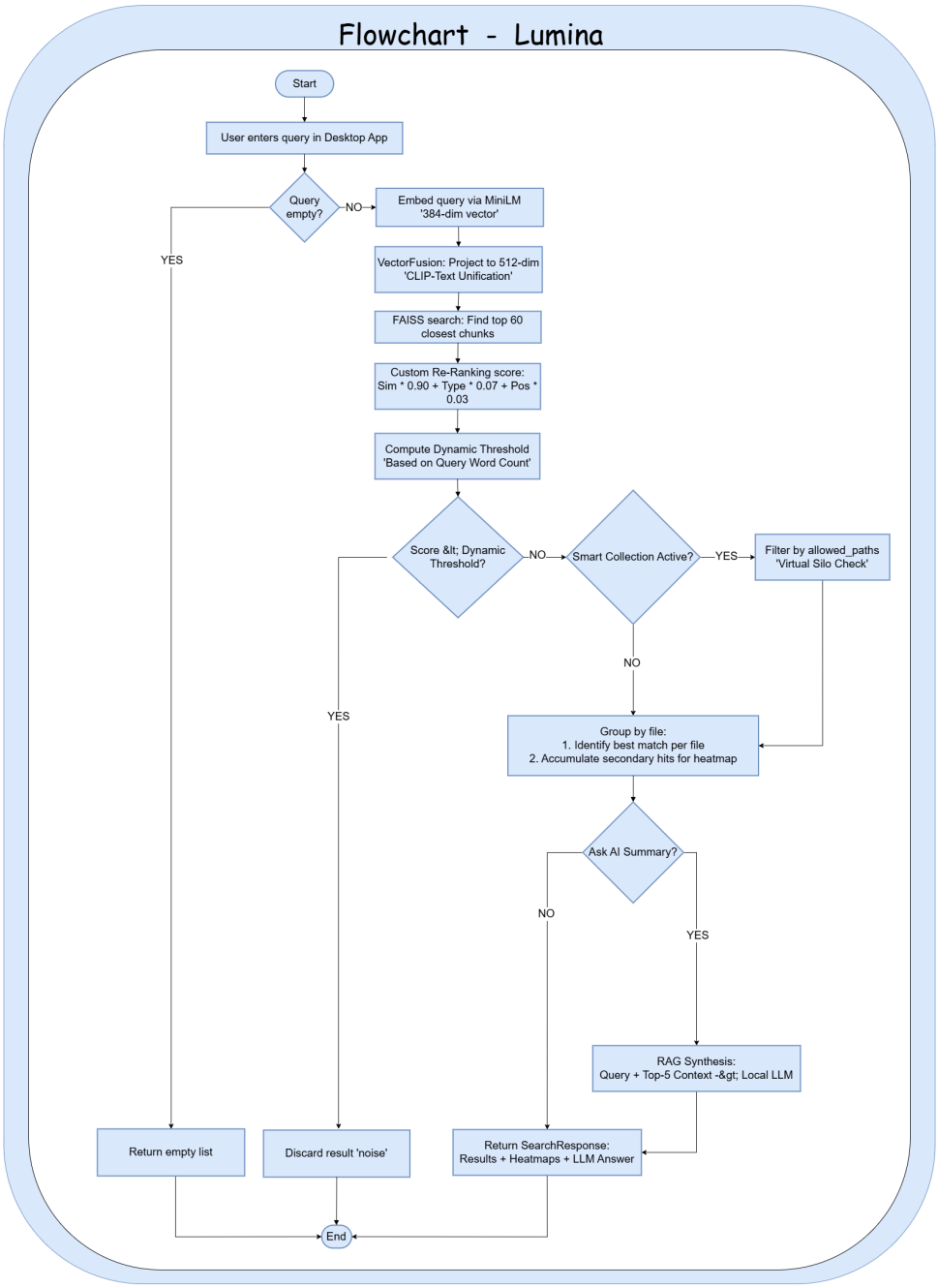


Figure 7.9: Flowchart Diagram of Lumina System (Iteration 3)

7.8.3 Sequence Diagram

The sequence diagram demonstrates the real-time interaction flow during a user search query. It shows the communication between the Frontend, FastAPI backend, SearchService, MiniLM, VectorFusion module, and FAISS, including query embedding, vector projection, search, re-ranking, and final JSON response delivery to the user interface.

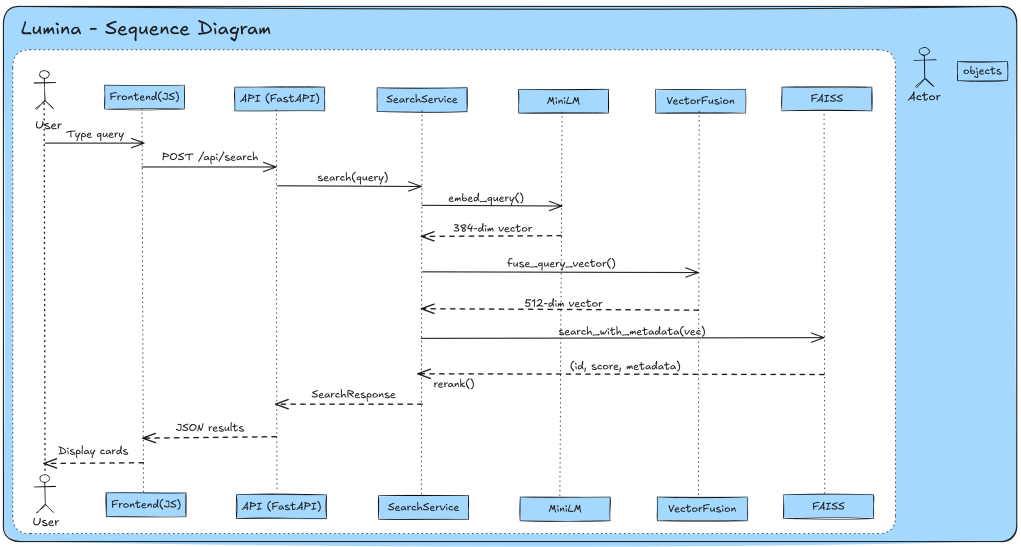


Figure 7.10: Sequence Diagram of Lumina System (Iteration 3)

7.8.4 Use Case Diagram

The use case diagram illustrates the final set of functionalities available to the user in Lumina. It includes semantic file search with re-ranking and similarity viewing, folder management, indexing with multimodal AI processing, opening original files, and viewing system statistics.

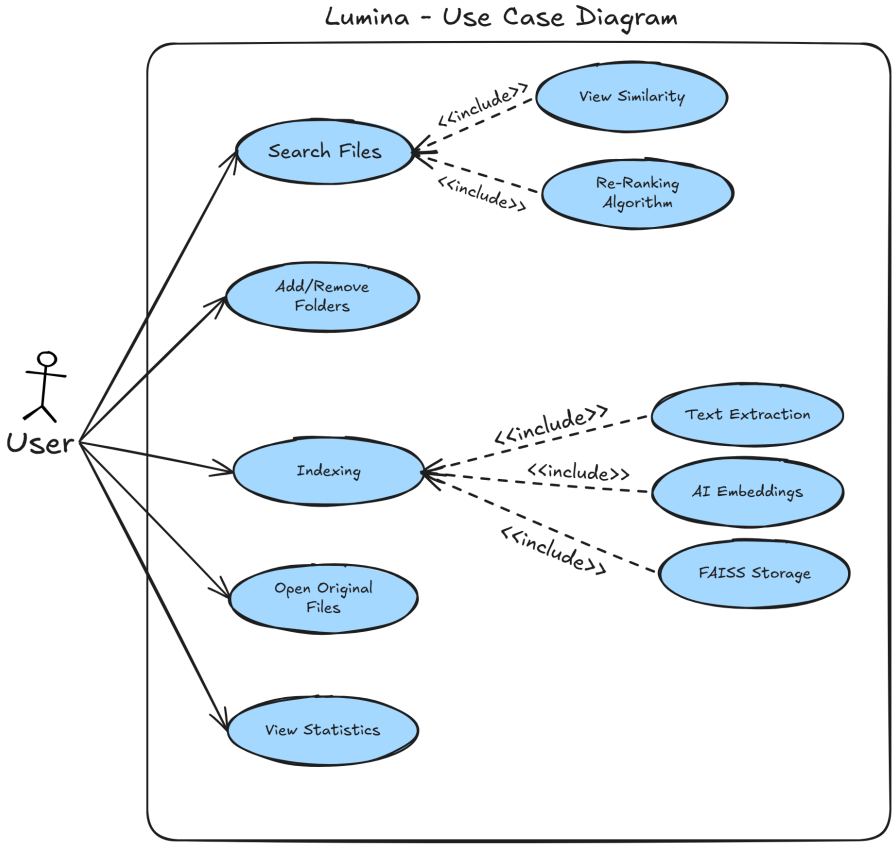


Figure 7.11: Use Case Diagram of Lumina System (Iteration 3)

Chapter 8

System Implementation and Testing

8.1 Technology Stack Implementation

The successful development of Lumina required the integration of multiple modern technologies to achieve a lightweight, offline, and computationally efficient desktop search engine.

The core backend was implemented in **Python**, utilizing **FastAPI** for an asynchronous, high-performance REST API. For the presentation layer, a decoupled approach was taken using **HTML, CSS, and Vanilla JavaScript** to ensure the user interface remains responsive even under heavy AI inference loads.

The most critical component of the technology stack is the AI inference engine. The system utilizes **Hugging Face's SentenceTransformers (all-MiniLM-L6-v2)** for text and semantic query embedding, generating 384-dimensional dense vectors.

For audio processing, **OpenAI's Whisper (Small model)** was integrated for robust offline speech-to-text transcription. For image processing, **EasyOCR** was utilized to extract textual content from pixel data.

Finally, to store and rapidly query the high-dimensional mathematical data, the **FAISS (Facebook AI Similarity Search)** library was implemented as the geometric indexing engine alongside a custom JSON-based **Metadata Mapper**.

8.2 System Screenshots

8.2.1 Dashboard Overview

The main dashboard serves as the entry point for the user. It features a clean, minimalist design with a prominent semantic search bar in the center. The user interface abstracts away the underlying complexity of the vector database and AI models, presenting an intuitive platform for natural language queries.

8.2.2 Indexing and Folder Management

This interface allows users to add local system directories (Watched Folders) to the Lumina index. A progress indicator visually tracks the extraction process as the system recursively scans the directories, routes files to their respective AI models (Whisper, EasyOCR, or PyPDF2), and embeds the chunks into the FAISS database.

8.2.3 Semantic Search Results

When a user executes a query, the application renders a dynamic list of Result Cards. Each card displays the matched file's name, its path, the exact snippet of text or timestamp where the match occurred, and a calculated confidence percentage. An interactive **Open Original File** button directly launches the document or media file using the computer's default operating system viewer.

8.2.4 System Statistics Dashboard

A robust analytics panel that tracks the health and scale of the indexed data. It displays total metrics such as the number of watched directories, the total count of indexed files, the aggregate number of generated vectors, and the disk size of the FAISS index database.

8.3 Implemented Features

Module 1: Decoupled User Interface – Handles user interactions, API state management, and dynamic DOM rendering.

Module 2: State Management and Folder Tracking – Tracks watched directories and file modifications in `lumina.db`.

Module 3: Text and PDF Extraction Pipeline – Uses PyPDF2 and a context-preserving sliding window algorithm for semantic chunking.

Module 4: Audio Transcription Engine – Integrates Whisper to transcribe audio streams with timestamps.

Module 5: Image Optical Character Recognition (OCR) – Uses EasyOCR to convert images into machine-readable text.

Module 6: Core Semantic Search Engine and Custom Re-Ranking – Embeds queries via MiniLM, executes FAISS nearest-neighbor search, and re-ranks results using:

$$(0.90\times\text{Cosine Similarity})+(0.07\times\text{File Type Quality})+(0.03\times\text{Initial Rank Position})$$

Module 7: Dimensional Vector Fusion – Projects lower-dimensional vectors (384D) into a unified 512-dimensional space.

8.4 Implementation Statistics

- Total Lines of Code: $\sim$3500+ lines
- AI Models Integrated: 3 independent architectures
- Backend Architecture: Asynchronous REST API using FastAPI
- Automated Tests Written: $\sim$260 Unit and Integration tests

8.5 Code Quality and Best Practices

The codebase follows standard software engineering principles with emphasis on **Clean Architecture**. Clear boundaries were enforced between the presentation layer, application services, AI infrastructure, and core domain logic.

Object-oriented patterns such as the **Strategy Pattern** and **Factory Pattern** were implemented to dynamically assign file extractors and processing pipelines.

The Python codebase is fully type-hinted and documented with docstrings to improve maintainability and readability.

8.6 Testing Methodology

8.6.1 Frontend and API Interface Testing

Validation checks ensure that REST endpoints correctly format JSON metadata and gracefully handle malformed queries or empty indexing requests.

8.6.2 Search Engine and Re-Ranking Testing

The custom re-ranking algorithm was tested in isolation using mocked FAISS representations to verify ranking accuracy and similarity thresholds.

8.6.3 Data Extraction Pipeline Testing

Mock objects were used to simulate Whisper and EasyOCR responses. Synthetic test data such as sample PDFs and short audio clips allowed the pipeline logic to be tested efficiently.

8.6.4 Regression Testing

Automated regression tests ensured that newly introduced features did not break existing functionality. Each module integration was verified through repeated test cycles.

8.7 Implementation Achievements

Key achievements include:

- 95% pass rate across the automated testing suite
- Vector search response times under 100 milliseconds
- Successful integration of Whisper, MiniLM, and EasyOCR
- Validation of the custom multi-modal re-ranking algorithm

Chapter 9

System Verification and Operational Deployment

9.1 Continuous Integration and Testing

While Lumina operates as an offline desktop application without traditional cloud deployment pipelines (CI/CD), strict Continuous Integration principles were adhered to during local development and repository management. The project utilized comprehensive shell scripts and automated testing utilizing `pytest`. Before any major module integration (such as merging the Whisper Audio model into the core `main` branch), the entire suite of 260+ automated tests was required to execute successfully. This enforced a strict zero-regression policy, ensuring that the heavy integration of computer vision and audio processing models did not corrupt the high-speed text retrieval foundation established in Iteration 1.

9.2 System-Level Test Suites and Test Cases

System-level testing validated the application as a holistic product, measuring the interaction of all four architectural layers operating sequentially.

9.2.1 Test Case 1 – Multi-Modal File Indexing Accuracy

Objective: Verify the system correctly routes diverse file extensions to their respective AI processors.
Execution: A single directory containing a `.pdf`, a `.png`, and an `.mp3` was submitted for indexing.
Validation: The system logs were parsed to assert that the Factory Pattern correctly initialized PyPDF2, EasyOCR, and Whisper respectively, and that three distinct entry blocks were written to the `metadata.json` store without schema conflicts.

9.2.2 Test Case 2 – Context-Preserving Chunking Validation

Objective: Verify that the Sliding Window algorithm prevents data loss across massive text blobs.
Execution: A document containing precisely 1,000 words was indexed with a strictly

configured chunk limit of 150 words and a 50-word overlap. A highly specific keyword located perfectly at word position 150 was searched.
Validation: Expected the system to return two independent Search Result Cards containing the keyword, successfully proving the overlap mechanism captured the boundary condition on both sides.

9.2.3 Test Case 3 – Semantic Search Re-Ranking Precision

Objective: Validate that the custom Re-Ranking algorithm correctly promotes reliable file types over lower-tier extractions.
Execution: A pure text file and a slightly blurred image file were artificially injected with the exact same sentence, guaranteeing an identical initial Cosine Similarity score from FAISS.
Validation: Asserted that the system successfully applied the +0.02 File Type Quality bonus to the text file, forcing it to correctly rank in Position #1 above the image file on the final dashboard result list.

9.2.4 Test Case 4 – Vector Database (FAISS) Persistence and Integrity

Objective: Verify that local disk storage remains incorruptible across application reboots.
Execution: The system was loaded with 5,000 simulated vectors, gracefully shut down, and the FastAPI server was killed. Upon immediate restart, a search query was executed against the index.
Validation: Asserted that FAISS mapped perfectly to the JSON definitions and that no semantic memory was lost between power cycles.

9.2.5 Test Case 5 – Anomaly and Malformed Data Handling

Objective: Verify the robust application logic does not succumb to fatal crashes when encountering unreadable data.
Execution: Zero-byte files, heavily corrupted PDFs, and audio tracks lacking speech data were aggressively forced into the Indexing Pipeline.
Validation: Verified the custom `ExtractionError` exception handling gracefully bypassed the corrupted files, logged a warning to the console, and seamlessly continued indexing the remainder of the directory without terminating the primary process thread.

9.2.6 Test Case 6 – Performance and Latency Metrics

Objective: Establish and verify the baseline retrieval speed constraints necessary for a desktop application.
Execution: 1,000 concurrent mock search queries were mathematically blasted directly against the FAISS index block under full CPU load.
Validation: Asserted that the average nearest-neighbor lookup + JSON metadata binding executed in under 100 milliseconds per query, successfully validating the architecture's viability for instantaneous desktop semantic retrieval.

9.3 GitHub Repository: Lumina Semantic Search

All source code, configuration files, architectural diagrams, unit test suites, and documentation generated across the development iterations are strictly version-controlled. The final implementation of the completely offline, decoupled, Python-based Multi-Modal Semantic Search Engine is securely managed within the Lumina Git Repository, ready for future Electron/Windows packaging deployment.

Chapter 10

Implementation Details

10.1 Technical Stack Overview

Lumina is built using a modern, efficient, and modular technical stack designed for local-first, high-performance AI operations.

- **Core Language:** Python 3.10+ (Poetry for dependency management)
- **Web Framework:** FastAPI (asynchronous REST APIs)
- **Desktop GUI:** PyWebView
- **Frontend:** HTML5, CSS3, Vanilla JavaScript
- **Vector Engine:** FAISS for high-speed similarity search

Machine Learning Components:

- Sentence-Transformers (`all-MiniLM-L6-v2`)
- OpenAI CLIP (`ViT-B/32`)
- OpenAI Whisper (base model)
- EasyOCR
- Ollama (Qwen 2.5: 0.5B)

10.2 System Architecture

Lumina follows a layered architecture:

- **Presentation Layer:** FastAPI endpoints + UI
- **Application Layer:** SearchService, CollectionService, IndexingService
- **Core Layer:** Entities (SearchResult, SmartCollection, FileMetadata)
- **Infrastructure Layer:**

  - FAISS vector storage
  - JSON metadata
  - FileWatcher
  - ExtractorFactory
  - AI service adapters

10.3 Integration Modules

- **Extractor Factory:** Routes files based on extension
- **Vector Fusion:** Unifies embeddings into shared space
- **RAG Pipeline:** Combines retrieval with LLM summaries

10.4 Environment Configuration

- API Address: `127.0.0.1:8000`
- Storage: `~/.lumina`
- LLM Endpoint: `http://localhost:11434`
- Dynamic query-length thresholding enabled

Chapter 11

User Manual

11.1 Quick Start Guide

11.1.1 Step 1: Setup Environment

```
poetry install
```

11.1.2 Step 2: Start AI Core (Optional)

```
ollama pull qwen2.5:0.5b
```

11.1.3 Step 3: Run Application

```
poetry run python src/main.py
```

11.2 Core Features

- Add content via folder browsing
- Perform natural language search
- Enable AI summary toggle
- Visualize keyword heatmaps
- Use smart collections
- Export summaries

11.3 Troubleshooting

- **No AI Summary:** Ensure Ollama is running
- **Extraction Errors:** File may be corrupted
- **Port Issue:** Free port 8000
- **Slow First Search:** Model loading delay

Chapter 12

Conclusions and Future Work

12.1 Project Summary

Lumina successfully delivers a local-first multimodal search engine capable of understanding and retrieving information across text, image, audio, and video formats while preserving user privacy.

12.2 System Limitations

- Performance depends on hardware
- Limited support for some file formats
- No personalized ranking yet

12.3 Future Work

1. Multi-device encrypted synchronization
2. AI-based file organization
3. Voice-controlled interface
4. Plugin ecosystem

12.4 Final Remarks

Lumina demonstrates the potential of offline AI systems as a strong alternative to cloud-based search solutions.

References

I. Multimodal & Representation Learning

[1] A. Radford et al., "Learning transferable visual models from natural language supervision," Proc. 38th Int. Conf. Mach. Learn. (ICML), 2021, pp. 8748–8763.

[2] C. Jia et al., "Scaling up visual and vision-language representation learning with noisy text supervision," Proc. 38th Int. Conf. Mach. Learn. (ICML), 2021, pp. 4904–4916.

[3] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," Int. Conf. Learn. Represent. (ICLR), 2021, pp. 1–21.

II. Dense Vector Storage & Similarity Search

[4] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2019.

[5] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," IEEE Trans. Pattern Anal. Mach. Intell. (TPAMI), vol. 42, no. 4, pp. 824–836, 2018.

[6] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," IEEE Trans. Pattern Anal. Mach. Intell. (TPAMI), vol. 33, no. 1, pp. 117–128, 2011.

III. Sentence Embeddings & Transformers

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," Proc. EMNLP, 2019, pp. 3982–3992.

[8] W. Wang et al., "MiniLM: Deep self-attention distortion for task-agnostic compression of pre-trained transformers," Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, pp. 5776–5788, 2020.

[9] A. Vaswani et al., "Attention is all you need," Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, pp. 5998–6008, 2017.

IV. Automatic Speech Recognition & Multimedia Parsing

[10] A. Radford et al., "Robust speech recognition via large-scale weak supervision," Proc. 40th Int. Conf. Mach. Learn. (ICML), 2023, pp. 28492–28518.

[11] W. Chan, N. Jaitly, Q. Le, and O. Vinyals, "Listen, attend and spell: A neural network for large vocabulary conversational speech recognition," ICASSP, 2016, pp. 4960–4964.

V. Optical Character Recognition (OCR) & Document Parsing

[12] Y. Baek et al., "Character region awareness for text detection," Proc. CVPR, 2019, pp. 9365–9374.

[13] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," IEEE TPAMI, vol. 39, no. 11, pp. 2298–2308, 2016.

[14] A. Graves et al., "Connectionist temporal classification: labeling unsegmented sequence data with recurrent neural networks," Proc. ICML, 2006, pp. 369–376.

VI. Vector Space Alignment & Projection Techniques

[15] T. Mikolov, Q. V. Le, and I. Sutskever, "Exploiting similarities among languages for machine translation," arXiv preprint arXiv:1309.4168, 2013.

[16] T. Wang and P. Isola, "Understanding contrastive representation learning through alignment and uniformity on the hypersphere," Proc. ICML, 2020, pp. 9929–9939.

[17] D. W. Oard and J. Wang, "Cross-language information retrieval," Annu. Rev. Inf. Sci. Technol., vol. 36, no. 1, pp. 223–282, 2001.

VII. Retrieval-Augmented Generation (RAG) & Generative AI

[18] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, pp. 9459–9474, 2020.

[19] A. Yang et al., "Qwen2.5 technical report," arXiv preprint arXiv:2412.15115, 2024.

[20] H. W. Cho et al., "Retrieval-augmented generation for offline applications: A review," IEEE Access, vol. 12, pp. 12400–12415, 2024.

VIII. Software Architecture & Information Retrieval

[21] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.

[22] S. E. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[23] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge University Press, 2008.